L'Architecture Client-Serveur et le Protocole HTTP

Thème principal : Architecture Web, Protocole HTTP, Requêtes/Réponses et Codes de Statut

Synthèse exécutive : Ce cours pose les fondements indispensables du fonctionnement du Web en détaillant l'architecture client-serveur et le protocole HTTP (HyperText Transfer Protocol). L'étudiant y apprendra comment un navigateur (client) initie de manière asymétrique des requêtes textuelles standardisées pour obtenir des ressources auprès d'un serveur web (Apache, Nginx). Le cours explique la nature 'stateless' (sans état) du protocole, la structure interne des échanges (lignes de commande/statut, en-têtes et corps), la différenciation stricte entre les méthodes GET et POST, ainsi que la classification des codes de statut HTTP (2xx, 3xx, 4xx, 5xx) indispensables au diagnostic et au débogage d'applications web.

1. L'Architecture Client-Serveur et le Principe Stateless

L'architecture client-serveur constitue le modèle standard de communication sur le réseau Web. Contrairement aux architectures Pair-à-Pair (P2P) où chaque nœud possède des privilèges équivalents, le modèle Web impose une asymétrie stricte : le client (navigateur web) prend l'initiative en envoyant une requête HTTP, tandis que le serveur attend de manière passive sur un port réseau, traite la demande et retourne une réponse. Une caractéristique fondamentale du protocole HTTP est sa nature 'stateless' (sans état). Le serveur ne conserve en mémoire aucune information sur les requêtes passées d'un client. Une fois la réponse envoyée, la transaction est terminée et le serveur 'oublie' le client. Si ce fonctionnement simplifie la conception des serveurs et offre une excellente scalabilité (gestion de millions de requêtes indépendantes), il interdit nativement les fonctionnalités nécessitant un suivi contextuel (panier d'achat, espace membre connecté). Pour surmonter cette contrainte, des mécanismes additionnels ont été créés : le serveur transmet un identifiant de session stocké sous forme de cookie par le navigateur, que ce dernier renvoie obligatoirement dans chaque en-tête de requête ultérieure.

- Client (Navigateur) : Logiciel actif qui prend l'initiative exclusive d'envoyer une requête HTTP pour obtenir une ressource.
- Serveur Web : Machine ou processus logiciel passif (ex: Apache, Nginx) écoutant sur un port dédié (80 pour HTTP, 443 pour HTTPS) et traitant les requêtes entrantes.
- Protocole Stateless (Sans état) : Propriété fondamentale de HTTP où chaque échange requête-réponse est totalement indépendant et isolé des précédents.
- Gestion de Session : Mécanismes complémentaires (cookies côté client, sessions côté serveur) introduits pour simuler un état persistant.

Cas concret enseignant : *Le professeur compare le protocole HTTP à un guichetier amnésique : chaque fois que vous vous présentez devant lui, même si c'était il y a deux secondes, il vous traite comme un inconnu. Pour qu'il se souvienne de vous, vous devez lui présenter un badge (le cookie de session) à chaque phrase. De plus, lorsqu'un étudiant charge la page d'accueil de l'IUT de Bayonne, le navigateur n'envoie pas une seule requête, mais plusieurs dizaines de requêtes HTTP simultanées pour récupérer séparément la page HTML, les feuilles de style CSS, les scripts JS et chaque image.*

2. Structure des Messages HTTP et Méthodes (GET vs POST)

Les messages HTTP transitent sous forme de blocs textuels structurés. Une requête HTTP se compose de : 1. La ligne de requête : contient le verbe/méthode (ex: GET, POST), l'URI (chemin de la ressource) et la version du protocole (ex: HTTP/1.1). 2. Les en-têtes (Headers) : métadonnées indiquant le domaine cible ('Host'), le type de navigateur ('User-Agent') ou les formats acceptés ('Accept'). 3. Une ligne vide obligatoire : sépare les en-têtes du contenu. 4. Le corps (Body) : optionnel, contient les données transmises au serveur. Le choix entre les méthodes GET et POST est critique pour la conception web : - GET est réservée à la lecture. Ses paramètres sont ajoutés à la fin de l'URL (query string: `?cle=valeur`). GET est limitée en longueur (environ 2048 caractères) et ne doit jamais altérer les données côté serveur. - POST est utilisée pour créer ou modifier des ressources (envoi de formulaire, création de compte). Les données sont encapsulées dans le corps de la requête, permettant de véhiculer de grands

volumes d'informations (fichiers, images) sans restriction de taille d'URL.

• Message HTTP Textuel : Format de communication lisible en clair composé d'une première ligne, d'en-têtes, d'une ligne vide et d'un corps optionnel.

• En-têtes (Headers) : Métadonnées sous forme de paires clé-valeur (ex: Host, User-Agent, Content-Type) fournissant le contexte du message.

• Méthode GET : Conçue pour consulter/récupérer des ressources. Les paramètres sont transmis directement dans l'URL (méthode sûre et idempotente).

• Méthode POST : Conçue pour soumettre des données modifiant l'état du serveur. Les données transitent de manière invisible dans le corps (body) de la requête.

Cas concret enseignant : *Le professeur met en garde contre l'utilisation de GET pour transmettre des identifiants : soumettre un mot de passe avec la méthode GET l'affiche directement en clair dans la barre d'adresse de l'écran, le sauvegarde dans l'historique du navigateur et l'enregistre dans les fichiers de logs du serveur. C'est une vulnérabilité majeure exposée au 'shoulder surfing' (regard par-dessus l'épaule).*

3. Classification des Codes de Statut HTTP

Chaque réponse envoyée par un serveur web débute par une ligne de statut comportant un code numérique à trois chiffres. Ce code indique immédiatement au client le résultat du traitement : - 2xx (Succès) : La requête a été reçue, comprise et acceptée (ex: `200 OK`). - 3xx (Redirection) : Indique que la ressource a déplacé son emplacement et nécessite une action supplémentaire du client (ex: `301 Moved Permanently`). - 4xx (Erreurs du Client) : Indique que le problème provient de la demande formulée par le client. Exemple : `404 Not Found` (ressource n'existe pas sur le serveur) ou `403 Forbidden` (accès non autorisé). - 5xx (Erreurs du Serveur) : Indique une défaillance interne du serveur lors de l'exécution d'une demande pourtant correcte. Exemple : `500 Internal Server Error` (crash d'un script PHP/Node.js, échec de connexion à la base de données). Pour un développeur, distinguer les familles 4xx et 5xx permet de cibler instantanément la zone d'intervention (code client/URL d'un côté, script backend/infrastructure serveur de l'autre).

• Ligne de Statut : Première ligne d'une réponse HTTP contenant la version du protocole, un code numérique à trois chiffres et un libellé textuel.

• Classes de Codes : 2xx (Succès), 3xx (Redirection), 4xx (Erreur Client), 5xx (Erreur Serveur).

• Erreur 4xx (Responsabilité Client) : La requête comporte une mauvaise syntaxe, cible une ressource absente ou manque d'autorisation (ex: 404 Not Found).

• Erreur 5xx (Responsabilité Serveur) : La requête du client est valide, mais le serveur a échoué lors du traitement interne (ex: 500 Internal Server Error).

Cas concret enseignant : *Le professeur raconte une mésaventure d'envergure : à ses débuts, une mauvaise configuration de son script PHP faisait planter la connexion à la base de données lors de la validation d'un formulaire. Le serveur renvoyait systématiquement une erreur 500 Internal Server Error. Les utilisateurs, effrayés par cette page brute, pensaient que le site avait été piraté alors qu'il s'agissait simplement d'un point-virgule manquant dans son code.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Monsieur, quelle est la différence concrète entre GET et POST pour envoyer un formulaire de connexion ?
Réponse du professeur : La différence est fondamentale à deux niveaux : la visibilité et la structure du message. Avec GET, les identifiants et mots de passe sont concaténés à l'URL (`login.php?user=matei&pass;=1234`), ce qui les rend visibles par tout le monde, les stocke dans l'historique du navigateur et dans les logs serveurs. Avec POST, les variables sont placées à l'intérieur du corps (body) de la requête HTTP, rendant la saisie invisible dans la barre d'adresse. De plus, GET est réservé à la consultation pure (idempotente) alors que POST est conçu pour effectuer des actions modifiant l'état du serveur.

■ Exercice & Démonstration d'Application

Énoncé : Analyser la requête HTTP brute suivante. Identifier la méthode utilisée, la ressource demandée, l'hôte cible, et expliquer le comportement du serveur avec le code de statut exact si la ressource demandée est absente : GET /images/logo.png HTTP/1.1 Host: iutbayonne.univ-pau.fr User-Agent: Mozilla/5.0

Résolution pas à pas :
1. Ligne de requête ('GET /images/logo.png HTTP/1.1') : - La méthode HTTP utilisée est 'GET' (demande de lecture/récupération de ressource sans modification de l'état du serveur). - La ressource demandée (URI) est le fichier '/images/logo.png'. - La version du protocole est HTTP/1.1. 2. Analyse de l'en-tête Host ('Host: iutbayonne.univ-pau.fr') : - Cet en-tête indique le nom de domaine cible sollicité. Il est obligatoire en HTTP/1.1 pour permettre au serveur de gérer plusieurs sites web virtuels sur la même adresse IP (Virtual Hosting). 3. Analyse de l'en-tête User-Agent ('User-Agent: Mozilla/5.0') : - Cet en-tête identifie le navigateur/client effectuant la requête. 4. Scénario d'absence de la ressource : - Si le fichier 'logo.png' n'existe pas dans le répertoire '/images/' sur le serveur web, la demande du client pointe vers un emplacement invalide. - La responsabilité incombant au client qui a formulé une URI erronée, le serveur renverra un code d'erreur de la classe 4xx, plus précisément le code de statut '404 Not Found'.

Résultat : Méthode : GET | Ressource demandée : /images/logo.png | Hôte cible : iutbayonne.univ-pau.fr | Code de réponse si absent : 404 Not Found (Erreur Client).

■ Auto-Évaluation Flash d'Examen

Q : Pourquoi qualifie-t-on le protocole HTTP de 'stateless' (sans état) et quelle est sa conséquence sur la gestion des sessions ?

R : HTTP est 'stateless' car le serveur ne conserve aucune mémoire des requêtes précédentes d'un même client. Chaque échange requête-réponse est totalement indépendant. La conséquence directe est l'impossibilité native de maintenir un utilisateur connecté ou de conserver un panier d'achat. Pour y remédier, on utilise des mécanismes de contournement comme les cookies et les sessions côté serveur.

Q : Quelle est la différence fondamentale de responsabilité entre une erreur HTTP de la série 4xx et une erreur de la série 5xx ?

R : Une erreur 4xx signale un problème imputable au client (URL mal orthographiée, absence d'autorisation, ressource introuvable comme 404), ce qui nécessite une correction de la demande ou du client. Une erreur 5xx indique une défaillance interne du serveur (crash de script PHP, panne de base de données comme 500) alors que la requête du client était valide, nécessitant une intervention technique sur le serveur.

Q : Pourquoi la méthode HTTP GET ne doit-elle jamais être employée pour transmettre un mot de passe ?

R : Parce que la méthode GET transmet les paramètres directement dans l'URL sous forme de chaîne de requête (query string). Le mot de passe apparaît en clair dans la barre d'adresse (exposition au shoulder surfing), est enregistré dans l'historique de navigation du client, et est sauvegardé dans les fichiers de logs d'accès du serveur web.